

**Mini Projet : Architectures parallèles et systèmes
distribués**

Système de fichiers distribué

LAKHLEF Abdelkrim

Table des matières

1	Introduction	3
2	Architecture générale	3
2.1	Client	3
2.2	Serveur de métadonnées	3
2.3	Serveurs de stockage	3
3	Protocole de communication	4
3.1	Actions supportées	4
4	Gestion des fichiers	4
4.1	Création	4
4.2	Lecture	4
4.3	Écriture	4
4.4	Suppression	4
5	Gestion de la concurrence	5
6	Conclusion	5

1 Introduction

Les systèmes de fichiers distribués sont utilisés pour stocker et accéder à des données réparties sur plusieurs machines, tout en offrant une abstraction proche d'un système de fichiers local. L'objectif de ce mini-projet est de comprendre les mécanismes fondamentaux d'un tel système à travers une implémentation simple mais fonctionnelle.

Le projet met l'accent sur :

- la séparation métadonnées / données,
- la communication réseau,
- la gestion de la concurrence via des verrous,
- la distribution des fichiers sur plusieurs serveurs.

2 Architecture générale

Le système repose sur trois types de composants :

- un **client**,
- un **serveur de métadonnées**,
- plusieurs **serveurs de stockage**.

2.1 Client

Le client fournit une interface en ligne de commande permettant à l'utilisateur d'exécuter des opérations telles que : **create**, **read**, **write**, **delete** et **list**. Il communique avec le serveur de métadonnées pour connaître la localisation des fichiers, puis contacte directement le serveur de stockage concerné.

2.2 Serveur de métadonnées

Le serveur de métadonnées joue un rôle central :

- il maintient une table associant chaque fichier à un serveur de stockage.
- il gère les verrous pour éviter les accès concurrents non contrôlés.
- il sélectionne un serveur de stockage lors de la création d'un fichier.

2.3 Serveurs de stockage

Chaque serveur de stockage conserve physiquement les fichiers dans un répertoire local. Il exécute les opérations de lecture, écriture et suppression.

3 Protocole de communication

La communication entre les composants se fait via des sockets TCP et des messages JSON. Chaque requête contient un champ "action" indiquant l'opération à effectuer.

3.1 Actions supportées

Les principales actions sont :

- CREATE
- READ
- WRITE
- DELETE
- LOCK / UNLOCK
- LIST

Chaque réponse inclut un code de statut indiquant le succès ou l'erreur de l'opération.

4 Gestion des fichiers

4.1 Création

Lorsqu'un client crée un fichier :

1. il contacte le serveur de métadonnées,
2. celui-ci choisit un serveur de stockage (via un hachage du nom),
3. le client envoie ensuite les données au serveur de stockage.

4.2 Lecture

Pour lire un fichier, le client obtient d'abord la localisation depuis le serveur de métadonnées, puis récupère le contenu directement depuis le serveur de stockage.

4.3 Écriture

L'écriture nécessite un verrou :

1. le client demande un verrou au serveur de métadonnées,
2. si le verrou est accordé, l'écriture est autorisée,
3. le verrou est libéré après l'opération.

4.4 Suppression

La suppression suit le même principe que l'écriture, avec verrouillage préalable pour éviter les conflits.

5 Gestion de la concurrence

Un mécanisme simple de verrouillage est implémenté :

- un fichier verrouillé ne peut être modifié par un autre client,
- les verrous sont gérés uniquement par le serveur de métadonnées,
- cela garantit une cohérence forte pour les écritures.

Cette approche est simple mais efficace pour un environnement à faible concurrence.

6 Conclusion

Ce mini-projet a permis de mettre en pratique les concepts essentiels des systèmes de fichiers distribués. Bien que simplifié, le système implémenté illustre clairement la séparation des responsabilités, la communication réseau et la gestion de la cohérence, qui sont au cœur des solutions industrielles plus complexes. Plusieurs améliorations peuvent être envisagées :

- réplication des fichiers sur plusieurs serveurs,
- tolérance aux pannes du serveur de métadonnées,
- authentification des clients,
- gestion de versions des fichiers,
- passage à une architecture totalement distribuée.